PYTHON PROGRAMMING — CLASS TEST

Name: ___Nnuta Matthew Ikechukwu_____Date: ___06/11/2026_____

Instructions: Answer all questions. Write code clearly where requested. Each topic contains five questions. Total: 80 questions across 16 topics | Files and Django excluded

Section 1: The Way of the Program

1. What is a computer program?

A computer program is a set of instructions that tells a computer what to do.

2. Explain the difference between source code and output.

Source code is the written instructions created by a programmer, while output is the result produced when the program runs.

3. Name two types of errors that can occur in a Python program.

Syntax errors and runtime errors.

4. What is the role of the Python interpreter?

The Python interpreter reads and executes Python code.

5. Write a Python statement that displays Welcome to Python.

```
print("Welcome to Python")
```

Section 2: Variables, Expressions and Statements

1. What is a variable? Give one valid Python variable name.

A variable is a name used to store data in a program. Example: student_name.

2. State the output of x = 4; x = x + 3; print(x).

The output is 7

3. Explain the difference between an expression and a statement.

An expression produces a value, while a statement is an instruction that performs an action.

4. Write an expression to calculate the area of a rectangle using length and width.

length * width

5. What is wrong with the variable name 2score?

A Python variable name cannot begin with a number.

Section 3: Functions

1. What is a function in Python?

A function is a reusable block of code that performs a specific task.

2. Write a function named greet that prints Hello.

```
def greet():
    print("Hello")
```

3. Explain the purpose of parameters in a function.

Parameters allow functions to receive data that they can use when performing their tasks.

4. What is the difference between defining and calling a function?

Defining a function means creating it, while calling a function means executing it.

5. Write a function call that passes 5 to a function named double.

double(5)

Section 4: Conditionals

1. What is the purpose of an if statement?

An if statement allows a program to make decisions based on whether a condition is true.

2. Write an if statement that prints Adult when age is at least 18.

```
if age >= 18:
    print("Adult")
```

3. Explain the difference between elif and else.

elif checks another condition if the previous condition is false, while else executes when all previous conditions are false.

4. What does the expression 7 > 3 and 2 > 5 evaluate to?

Output: False, because both conditions must be true when using and.

5. Write a conditional that prints Even if a number is divisible by 2 and Odd otherwise.

```
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Section 5: Recursion

1. What is recursion?

Recursion is a programming technique where a function calls itself to solve a problem.

2. What is a base case in a recursive function?

A base case is the condition that stops a recursive function from calling itself.

3. Explain what may happen if a recursive function has no reachable base case.

The function may continue calling itself until Python raises a RecursionError.

4. Write a recursive function that returns the factorial of a non-negative integer.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

5. State one difference between recursion and iteration.

Recursion uses a function calling itself, while iteration repeats instructions using loops.

Section 6: Fruitful Functions

1. What is a fruitful function?

A fruitful function is a function that returns a value when executed.

2. How does return differ from print inside a function?

return sends a value back to the caller, while print only displays information on the screen.

3. Write a function square(n) that returns n multiplied by itself.

```
def square(n):
    return n * n
```

4. What value is assigned to result after result = max(3, 9)?

The value assigned to result is 9.

5. Write a function that accepts two numbers and returns their average.

```
def average(a, b):
    return (a + b) / 2
```

Section 7: Iteration

1. What is iteration in programming?

Iteration is the process of repeating a block of code multiple times.

2. Write a for loop that prints numbers 1 through 5.

```
for i in range(1, 6):
    print(i)
```

3. Explain one difference between for and while loops.

A for loop is commonly used to iterate over a sequence, while a while loop repeats as long as a condition is true.

4. What is the purpose of break in a loop?

The break statement immediately terminates a loop.

5. Write a while loop that counts down from 3 to 1.

```
x = 3
while x >= 1:
    print(x)
    x -= 1
```

Section 8: Strings

1. What is a string in Python?

A string is a sequence of characters enclosed in single or double quotation marks.

2. What is the output of print("Python"[0])?

Output: P.

3. Explain what string slicing means.

String slicing is the process of extracting a portion of a string using index positions.

4. Write Python code that converts "hello" to uppercase.

```
print("hello".upper())
```

5. What is the result of len("coding")?

Output: 6.

Section 9: Lists

1. What is a Python list?

A list is an ordered collection of items that can be changed after creation.

2. Write a list containing three programming languages.

```
languages = ["Python", "Java", "JavaScript"]
```

3. How do you access the second element of a list named items?

Use items[1].

4. Explain the difference between append() and remove().

append() adds an item to the end of a list, while remove() deletes a specified item.

5. What will print([10, 20, 30][-1]) display?

Output: 30.

Section 10: Dictionaries

1. What is a Python dictionary?

A dictionary is a collection of key-value pairs used to store and organize data.

2. Create a dictionary with keys name and age.

```
student = {"name": "John", "age": 20}
```

3. How do you retrieve the value stored under the key name?

Use student["name"].

4. What happens when an existing dictionary key is assigned a new value?

The previous value is replaced with the new value.

5. State one difference between a dictionary and a list.

A list uses numerical indexes to access elements, while a dictionary uses keys.

Section 11: Tuples

1. What is a tuple in Python?

A tuple is an ordered collection of items that cannot be changed after creation.

2. Write a tuple containing the numbers 2, 4 and 6.

```
numbers = (2, 4, 6)
```

3. State one key difference between a tuple and a list.

A list is mutable, while a tuple is immutable.

4. What is the output of print((5, 8, 11)[1])?

Output: 8.

5. Can you directly change an element of a tuple? Explain.

No, because tuples are immutable and their elements cannot be directly modified after creation.

Section 12: Classes and Objects

1. What is a class in Python?

A class is a blueprint used to create objects.

2. What is an object?

An object is an instance of a class that contains data and can perform actions.

3. Write a simple class named Student with no attributes or methods.

```
class Student:
    pass
```

4. Explain the relationship between a class and an instance.

A class is a blueprint, while an instance is an individual object created from that class.

5. What is the purpose of the __init__ method?

The __init__ method initializes an object's attributes when the object is created.

Section 13: Classes and Functions

1. How can a function be used to create an object of a class?

A function can create an object by calling the class constructor and returning the created instance.

2. Write a function make_student() that returns an instance of a Student class.

```
def make_student():
    return Student()
```

3. What is the difference between a function defined outside a class and one defined inside it?

A function outside a class is independent, while a function inside a class is called a method.

4. Explain why a function might accept an object as an argument.

A function accepts an object as an argument to access or work with its attributes and methods.

5. Write a function that accepts an object with a name attribute and prints its name.

```
def show_name(obj):
    print(obj.name)
```

Section 14: Classes and Methods

1. What is a method in Python?

A method is a function defined inside a class that performs an action related to that class.

2. What does self usually refer to in an instance method?

self refers to the current instance of the class.

3. Write a class Dog with a method bark() that prints Woof.

```
class Dog:
    def bark(self):
        print("Woof")
```

4. How do you call the bark method on an object named pet?

Use pet.bark().

5. Explain the difference between an instance attribute and a method.

An instance attribute stores data belonging to an object, while a method defines an action the object can perform.

Section 15: Inheritance

1. What is inheritance in object-oriented programming?

Inheritance is a feature that allows one class to acquire attributes and methods from another class.

2. Write a class Car that inherits from a class Vehicle.

```
class Car(Vehicle):
    pass
```

3. State one benefit of inheritance.

Inheritance promotes code reuse and reduces code duplication.

4. What is method overriding?

Method overriding occurs when a subclass provides its own implementation of a method inherited from its parent class.

5. Explain the purpose of super() in a subclass.

super() allows a subclass to access methods or initialize attributes from its parent class.

Section 16: Analysis of Algorithms

1. What does algorithm efficiency mean?

Algorithm efficiency measures how well an algorithm uses time and memory to solve a problem.

2. What is time complexity?

Time complexity describes how an algorithm's running time changes as the input size increases.

3. Explain the meaning of O(n) in simple terms.

O(n) means the running time increases proportionally with the size of the input.

4. Which usually scales better for large n: O(n) or $O(n^2)$? Explain.

O(n) scales better because its running time grows more slowly than $O(n^2)$ as the input size increases.

5. State one reason why space complexity matters.

Space complexity matters because it determines how much memory an algorithm requires.